Scope-Aware Security Triage: A Java and Semgrep Provenance Pilot

SAURAV K. ARYAL, Howard University, USA

An empty static-analysis result is ambiguous when the intended files, rules, or revision were not analyzed. We examine this evidence-consumption problem in a small Java provenance pilot. Six authored path-reading fixtures are compiled and executed under four local probes each; four fixtures disclose an outside sentinel in at least one probe. Actual Semgrep 1.183.0 scans use one authored taint rule. We compare a deterministic consumer that treats no finding as clearance with a consumer that first checks revision, file inclusion, source hashes, and required-rule identity. Under partial current-revision analysis, the unqualified consumer clears three files, including two with observed disclosure; scope qualification instead escalates the three unanalysed files. Under a simulated omitted-rule condition, four disclosing files are cleared without qualification, while the qualified consumer escalates all six files. The clean current scan flags five files and has no observed false clearance. The stale-report control over-reports rather than misses findings, so it demonstrates no false-clearance benefit from qualification and exposes an abstention cost. These results establish a reproducible provenance-harness behavior on authored fixtures. They do not establish vulnerability-detector accuracy, security-agent performance, or effectiveness on public repository histories.

Additional Key Words and Phrases: static analysis, provenance, security triage, Semgrep, reproducibility

1 Introduction and Background

Static-analysis output describes an invocation, with particular source files, rules, tool versions, and analysis conditions. A consumer that interprets the absence of a finding as a clearance can erase those qualifications. This is especially problematic when reports are reused after edits, a scan covers only part of the intended change, or the required rule did not run. The underlying analyzer need not be defective for its result to become inapplicable to the consumer's question.

Existing work already combines language models with static analysis. IRIS infers taint specifications and performs contextual analysis for repository-level vulnerability detection [3]. A recent preprint studies model deference to static-analysis assurances, including manipulated evidence [4]. Our pilot neither reproduces those model studies nor proposes another vulnerability model. Its narrower question concerns a deterministic evidence interface: can explicit invocation scope prevent a specific no-finding-to-clearance error without pretending that scope validates the analyzer's semantics?

The relevant metadata are not a new interchange-format invention. SARIF already standardizes static-analysis output [1]. GitHub's CodeQL documentation describes tool, artifact, and rule information, including fields whose availability depends on configuration [2]. Those formats provide a foundation for a future adapter. Here, a trusted local harness records the fields directly and compares them with the current intended target. The pilot therefore examines a consistency gate, not a new standard or a cryptographic provenance system.

We use actual Semgrep with a deliberately small local taint rule and actual Java execution to avoid treating source patterns as observed effects. Semgrep documents source-to-sink taint rules [6]; the experiment pins release 1.183.0 [7]. The artifact repository is https://github.com/Saurav-K-Aryal/scopeledger-pilot. Our result is a controlled engineering demonstration. Careful public-code evaluation remains necessary, particularly because vulnerability benchmarks can reward unrelated features or fail to distinguish vulnerable and patched programs [5].

Author's Contact Information: Saurav K. Aryal, Howard University, Washington, DC, USA.

2 Methods

2.1 Fixtures and execution oracle

The harness authors six small Java programs that accept a root directory and an input path, then read a file. The variants use direct path resolution, a `File` join, normalized path containment, a textual `..` filter, a constant filename, and a normalized string-prefix check. A local Git repository first contains a checked-normalization baseline for all six names, then a second committed revision contains these variants. This supplies a real revision change while keeping the programs inspectable. The variants share the path-reading domain; calling them separate repository samples or learned generalization would be inappropriate.

The current revision is compiled with `javac`. Four inputs are run against each class: a legitimate inside path, a parent-directory escape, an absolute outside path, and a sibling directory whose name shares the root's string prefix. The sandbox contains one public sentinel inside the root and two private sentinels outside it. A file is labeled *observed disclosing* only if at least one execution prints a private sentinel. There are 24 probe executions, not 24 independent programs. Non-disclosure in these probes is not an exhaustive safety proof; symbolic links, platform differences, and other inputs are outside this oracle.

This execution oracle matters for the prefix case. Testing a path's string representation with a root-prefix comparison is different from checking normalized path containment. The sibling-path probe exercises that distinction. It also matters for joins: the absolute-path File-join probe throws an exception in the observed environment, while other probes disclose the outside sentinel. We preserve execution failures instead of counting every attack-shaped input as a disclosure.

2.2 Analyzer evidence and conditions

Actual Semgrep scans use `mode: taint`, with `args[1]` as a source and `Files.readAllBytes(...)` as a sink. No sanitizer is specified. This authored rule is not a validated security rule pack and does not encode complete path-containment semantics. JSON reports record version 1.183.0, no analysis errors, actual invocation arguments, and per-file findings. The built-in toy lexical mode available in the development runner is excluded from every numerical result in this article.

The harness captures a report manifest containing the Git revision, invoked file list, current source hashes for those files, rule identities, and findings. Four conditions use the same six current target files and the same execution-derived labels. *Clean current* scans every current file. *Stale previous revision* reuses the earlier actual scan. *Partial current* scans three files sampled once with seed 2026: DirectResolve, CheckedNormalize, and StringPrefix. *Omitted rule pack* is an explicit simulated empty-evidence condition: the manifest has no required rule or findings, and no Semgrep command is executed for that condition. It is not a failed scanner run or a claim that Semgrep silently omitted a rule.

2.3 Consumers and measurements

The unqualified consumer returns *finding* for a file present in the finding list and *clear* otherwise. The scope-qualified consumer first requires four checks: matching current revision, file included in the invocation, matching source hash, and the required local rule present. If any fails it returns *insufficient evidence*; otherwise it uses the same finding/clearance rule. This is a conservative deterministic gate, with no language-model calls, recovery action, or analyst judgment.

We count clearances, false clearances against observed disclosure, findings, and insufficient-evidence decisions. "Clear" is the harness consumer's action, not a guarantee of software safety. All conditions are paired transformations

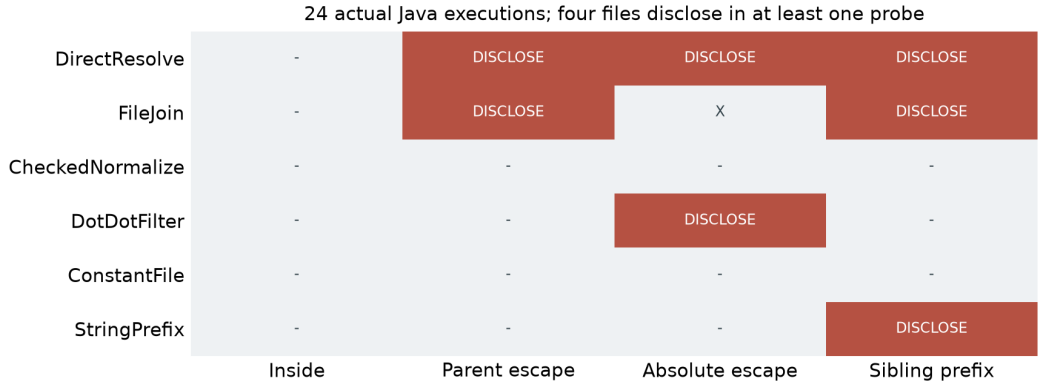


Fig. 1. Actual compiled-Java probe results. Filled cells indicate observed private-sentinel disclosure; the cross is an execution exception. Other cells show no disclosure for that input, which does not establish general safety.

of the same six files; no population confidence interval is warranted. We retain both a positive control where scope is current and a negative utility control where stale evidence already raises warnings. Source and result SHA-256 hashes are included with the figure data to bind the paper to the actual run.

3 Results

3.1 Observed effects and clean evidence

Four files disclose an outside sentinel: DirectResolve, FileJoin, DotDotFilter, and StringPrefix. The direct-resolution fixture discloses under three probes; FileJoin under two; DotDotFilter and StringPrefix under one each. CheckedNormalize and ConstantFile show no disclosure under the four probes. Figure 1 reports the individual outcomes rather than replacing execution with an assumed vulnerable/safe label.

The clean current Semgrep report flags five of six files, including all four observed-disclosing files and CheckedNormalize. Both consumers return five findings and clear only ConstantFile. Thus there is no observed false clearance for scope qualification to improve in this condition. The CheckedNormalize warning is consistent with a rule that has no sanitizer specification; it is not evidence of a newly discovered defect in that fixture. The real Semgrep run also flags the string-prefix fixture, so the toy scanner's semantic miss must not be attributed to this run.

3.2 Incomplete evidence and escalation

With only three current files analyzed, the unqualified consumer clears the other three: FileJoin, DotDotFilter, and ConstantFile. Two of these disclose the target sentinel. The qualified consumer retains the three available findings and marks the three omitted files insufficiently analyzed. Observed false clearances therefore change from two to zero, with three escalations and zero clearances. This is a trade between an invalid clearance and explicitly unresolved work, not evidence that qualification discovers the two defects independently.

In the simulated omitted-rule condition, the unqualified consumer clears all six files, including the four observed-disclosing files. The qualified consumer escalates all six because required-rule evidence is missing. Again, no new semantic detection is added. The gate refuses to turn empty evidence into a conclusion about the requested analysis. Figure 2 shows false-clearance counts and insufficient-evidence counts separately so that abstention is visible.

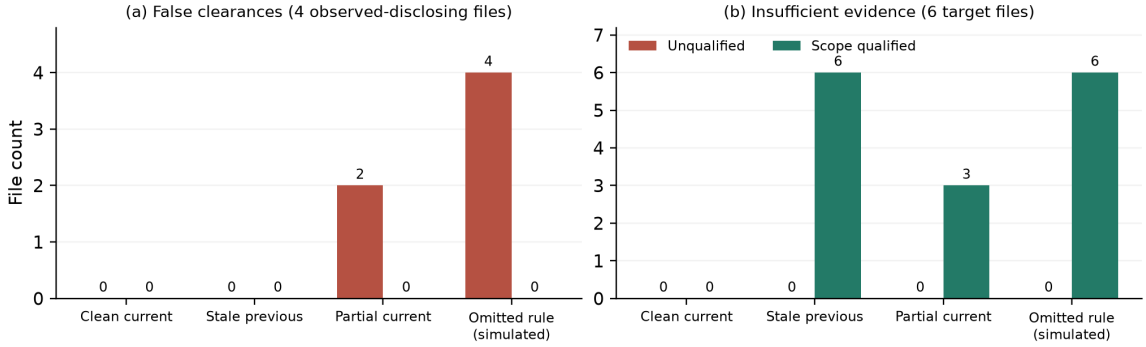


Fig. 2. Consumer outcomes per condition over the same six target files. Left: observed false clearances, among four files with a demonstrated disclosure. Right: insufficient-evidence decisions, among all six files. Zero false clearance can accompany substantial unresolved work.

3.3 Stale evidence as a negative utility control

The previous-revision scan flags all six earlier checked-normalization programs because the local rule does not specify the guard as a sanitizer. Reusing this report therefore produces six unqualified findings and no clearances. The qualified consumer instead returns six insufficient-evidence outcomes because the revision differs. Five files also have differing source hashes; CheckedNormalize is unchanged but is still rejected by the whole-revision check. Both consumers have zero observed false clearances.

This control gives no false-clearance reduction attributable to the scope gate. It instead exposes the conservative gate's utility cost: even warnings and unchanged content become unresolved when global revision qualification fails. A future file- and dependency-sensitive policy could retain valid evidence more selectively, but that policy has not been tested here. We do not replace this negative result with a more favorable stale example.

3.4 Limits of the evidence

The trust boundary includes the harness's own Git, build, invocation, and hash capture. Forged manifests, malicious analyzers, and incomplete extraction are not tested. Correct metadata cannot establish that a rule models a weakness adequately, and limited dynamic probes cannot label all vulnerabilities or prove all negatives. Six authored files in one language and one weakness family do not estimate operational miss rates, alert precision, or analyst workload. There are no public CVEs, new vulnerability discoveries, natural repository histories, or LLM triage outcomes in this pilot.

4 Future Work

A useful next study should adapt mature analyzer output rather than replace its semantics. The initial milestone is a reproducible set of approximately twenty public vulnerable/fixed Java pairs with pinned toolchains, query configurations, and independently checked target effects. Later expansion should retain repository and related-revision grouping, a temporal holdout where feasible, and documented build exclusions. The distinction between the original IRIS paper and a maintained code snapshot should be preserved through exact source revisions.

The main experiment would compare unqualified hybrid triage, deterministic scope checking, and budgeted targeted reanalysis on the same locked cases. Faults should include stale reports, omitted files, incomplete extraction, and missing query configurations. Clean current evidence remains essential: refusing everything trivially avoids clearance

while being unusable. Primary outcomes should report false clearance together with retained recall, unresolved cases, recovery success, and analysis cost. An existing baseline with zero false clearance requires absolute differences, rather than a relative-reduction headline.

Ablations should remove revision binding, file inclusion, source hashes, and rule identity separately. The stale-control result motivates comparing whole-revision rejection with a conservative dependency-sensitive alternative. Specialist review must distinguish an actual demonstrated defect from an analyzer warning, and analyst-time studies should be exploratory until recruitment and measurement plans are justified. A one-year effort could devote the first quarter to reproduction and corpus qualification, the next two to adapters and a locked fault suite, and the final quarter to evaluation and independent replay. None of those milestones is claimed as completed by this pilot.

5 Conclusion

On six authored Java fixtures, actual Semgrep output becomes misleading when a consumer drops its file and rule scope. Explicit scope checks prevent the observed false clearances in partial-analysis and omitted-rule controls by returning insufficient evidence. Clean current analysis has no observed false clearance, while stale evidence over-reports and yields no clearance benefit from qualification. The result supports an auditable evidence interface and a carefully controlled repository study. It does not establish semantic completeness of a security analyzer or the effectiveness of a deployed security agent.

Artifact preparation. Code, experiment execution, and manuscript preparation used Codex assistance under the author's direction.

References

[1] Michael C. Fanning and Laurence J. Golding (Eds.). 2020. *Static Analysis Results Interchange Format (SARIF) Version 2.1.0.* OASIS. https://docs.oasis-open.org/sarif/sarif/v2.1.0/os/sarif-v2.1.0-os.html OASIS Standard, March 27, 2020.

[2] GitHub. 2026. CodeQL CLI SARIF Output. https://docs.github.com/en/code-security/reference/code-scanning/codeql/codeql-cli/sarif-output Official documentation; accessed October 31, 2026.

[3] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. https://arxiv.org/abs/2405.17238v3 Author manuscript, arXiv:2405.17238v3.

[4] ZhiHong Liu and Shuyao Gao. 2026. Trusting the Tool: Asymmetric Deference of LLM Vulnerability Auditors to Static-Analysis Evidence. https://arxiv.org/abs/2610.32876 Preprint, arXiv:2610.32876; submitted for review.

[5] Niklas Risse and Marcel Böhme. 2024. Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection. In *33rd USENIX Security Symposium*. USENIX Association, 4247–4264. https://www.usenix.org/conference/usenixsecurity24/presentation/risse

[6] Semgrep. 2026. Taint Analysis Overview. https://docs.semgrep.dev/writing-rules/data-flow/taint-mode/overview Official documentation; accessed October 31, 2026.

[7] Semgrep contributors. 2026. Semgrep Release v1.183.0. https://github.com/semgrep/semgrep/releases/tag/v1.183.0 Official release, October 29, 2026.